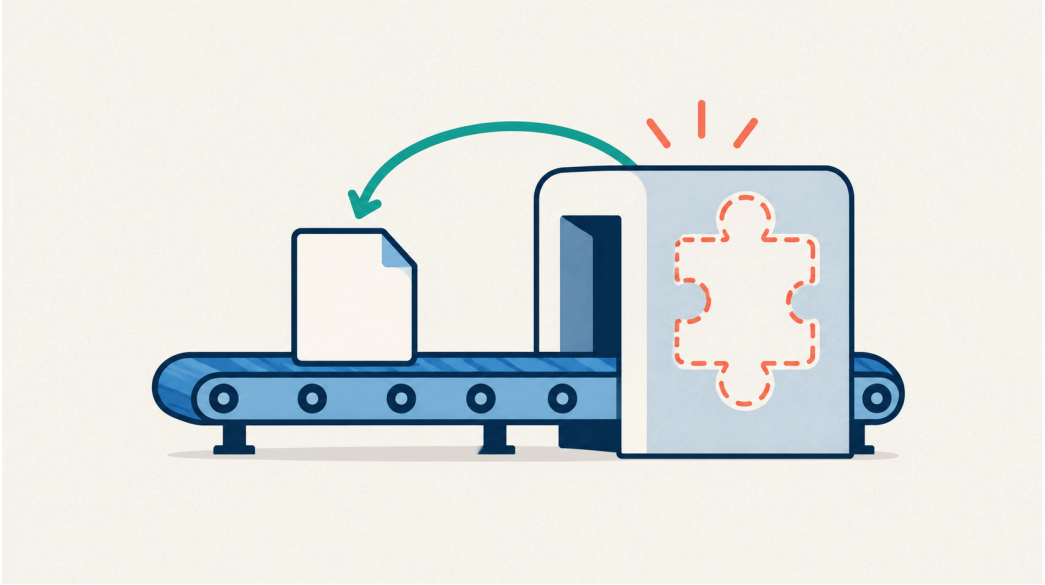
The Missing Test Suite

Where the bottleneck goes when code is cheap

Daniel Fridljand · danielfridljand.de

TNG Technology Consulting
4 December 2026



Early in the project, we had to decide: workflow or agent?

Worth a talk of its own: Workflow or Agent.
Today we take it as fixed: we built a workflow.

Code is cheap now.
So what becomes the bottleneck?

On a support automation I built: the experts who could tell whether a reply was right.

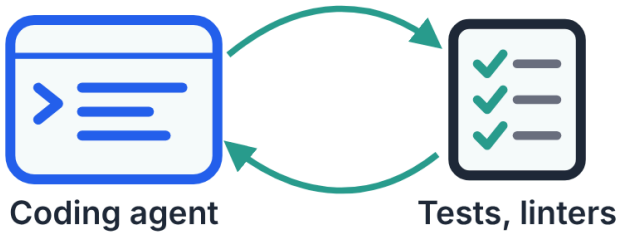
Why coding agents work

1. **Software is familiar.** Syntax, idioms and libraries are patterns the model has seen countless times in training.
2. **Every attempt gets a verdict.** Tests, linters and error messages tell the agent whether it worked, so the agentic loop runs without a human in it.

A reply to a customer has no test suite

- Code: the verdict comes **on every attempt**, for free.
- A reply: the verdict comes **when an expert reads it**.
- The model hasn't seen your escalation rules or your customers' records either.

Code



Reply to a customer



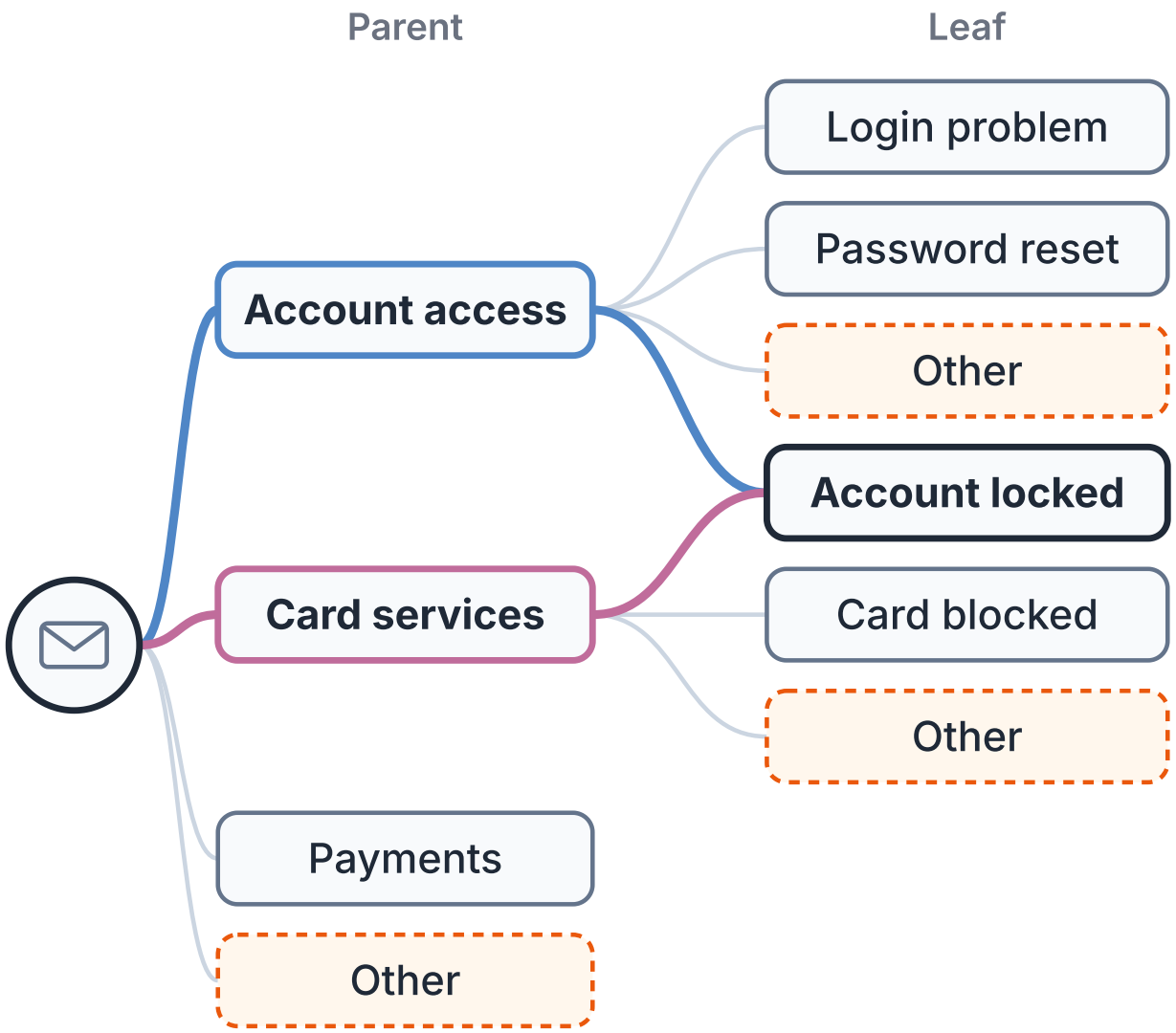
The running example

Customer · 2026-03-12 09:41

Hi, I'm Alex Carter and I bank with your Riverton branch. My account number is NSB01234 and I can't get in any more; it keeps saying the account is locked. Can you help me get back in?

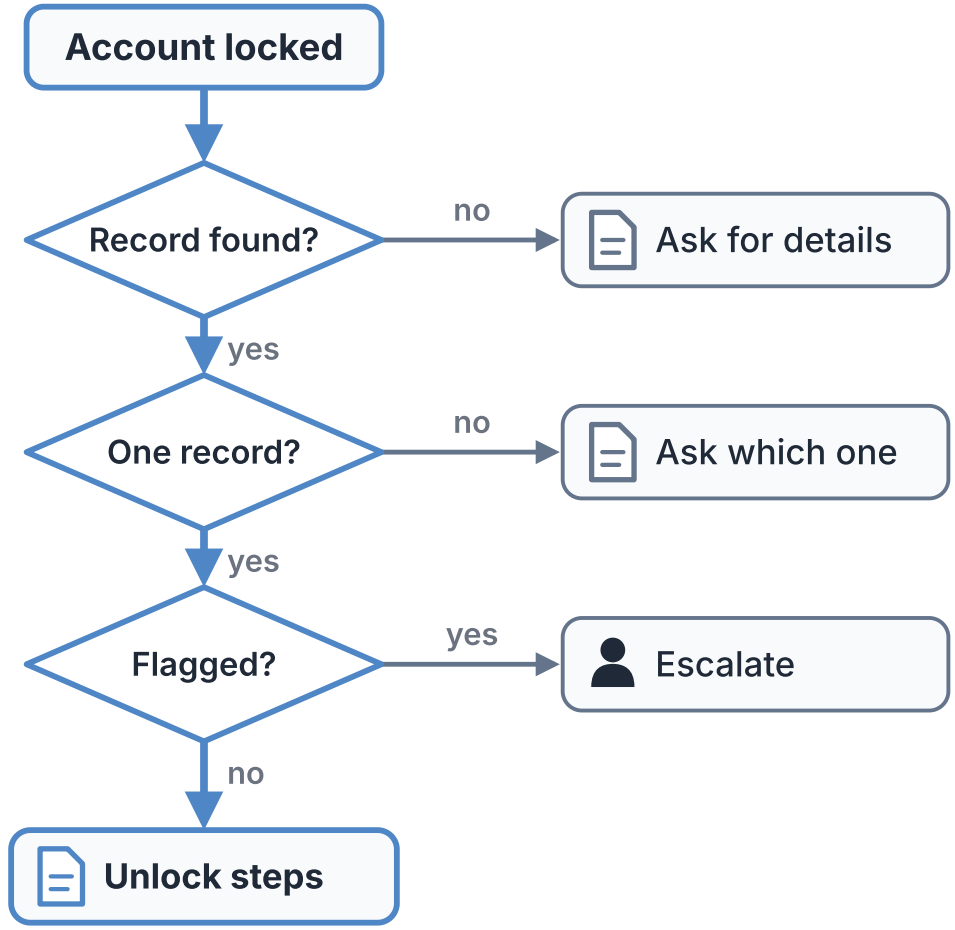
Invented to protect the client; the project behind it is real.

The workflow, step one: which kind of request is it?

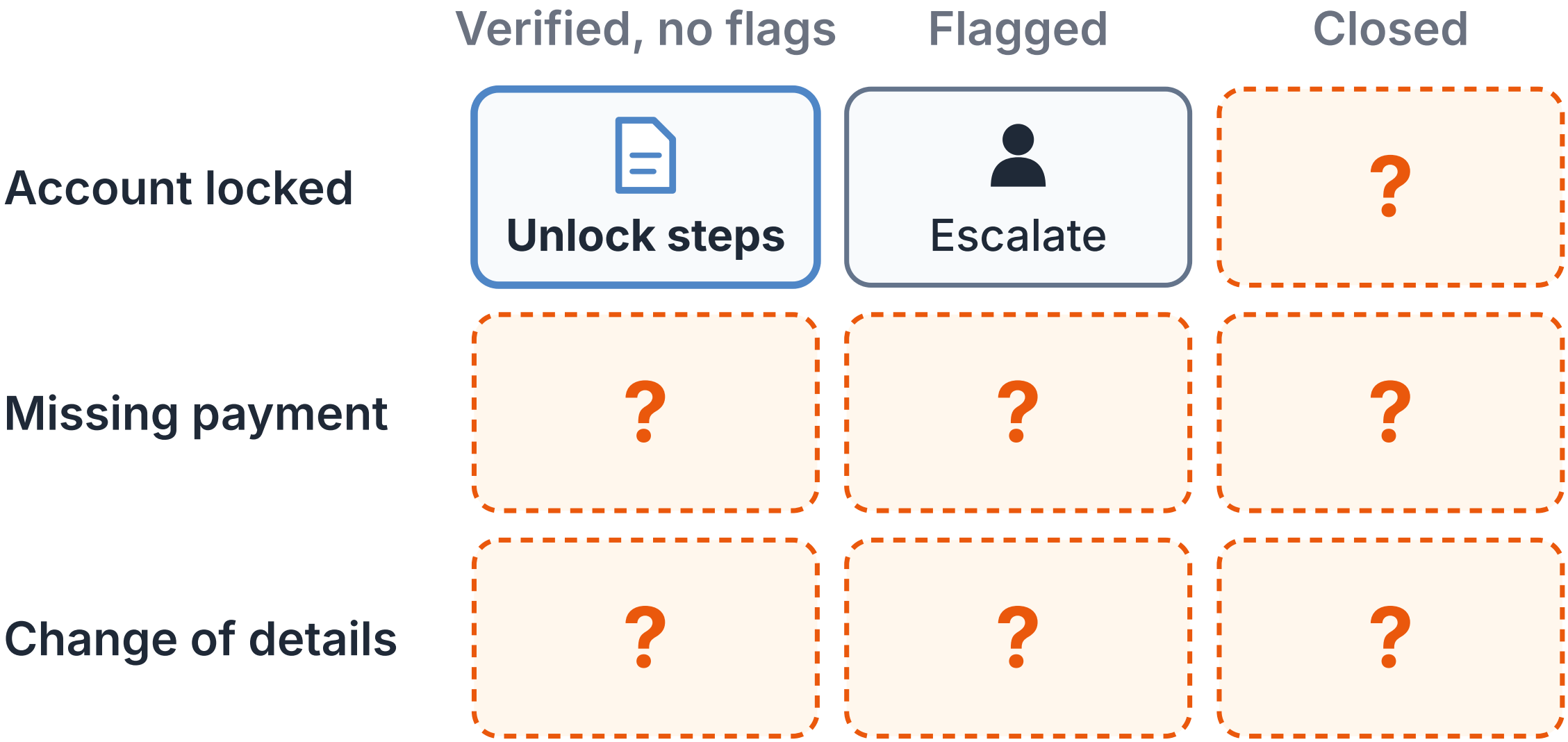


Then code decides what happens

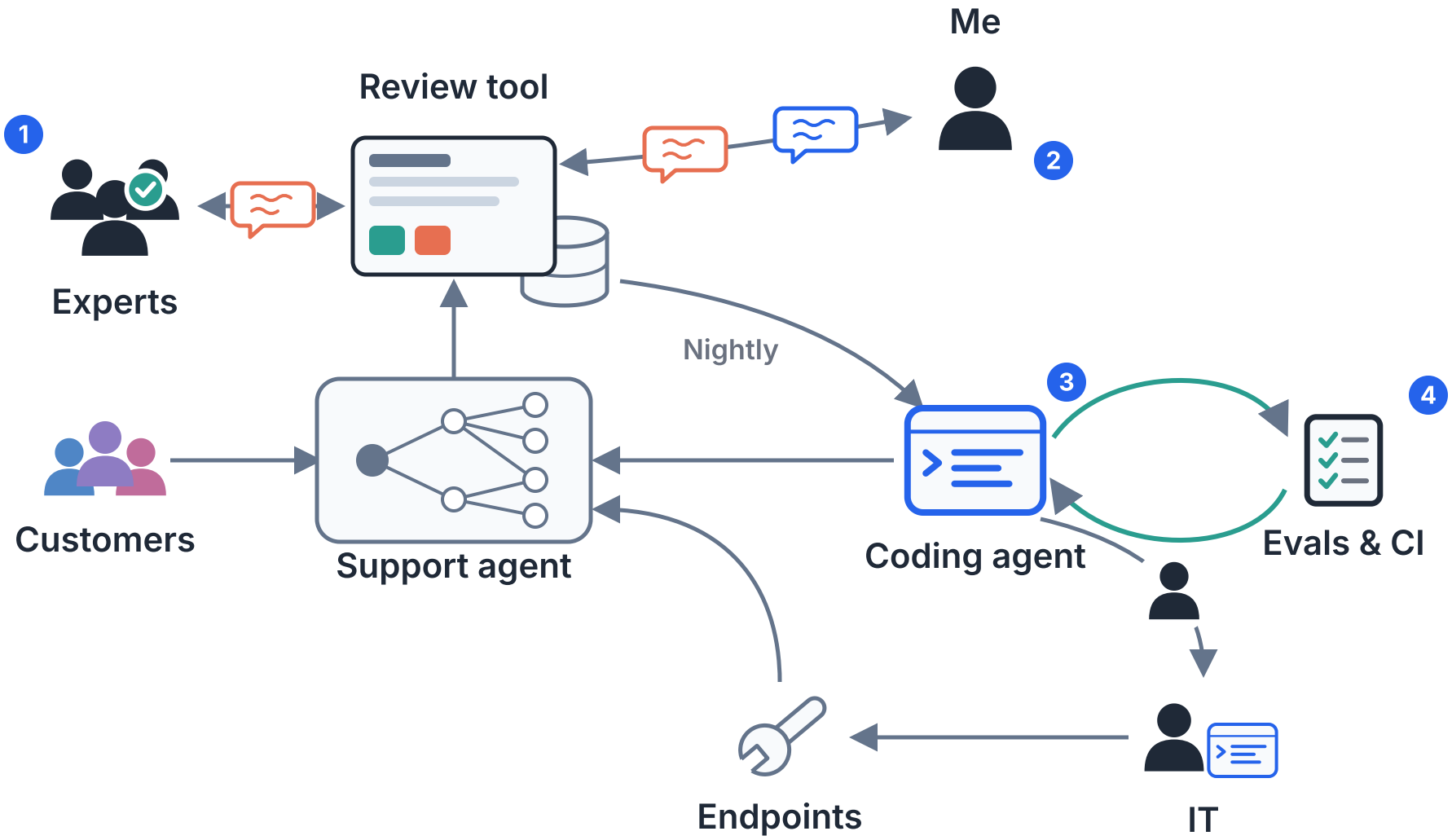
- The model **reads** the message: the intent and the account number.
- Code **decides** the order: look up, check the flag, then unlock or escalate.
- The security check comes first, every time, and the reply is a fixed template.



A branch for every cell: a steady stream of small changes



The whole system

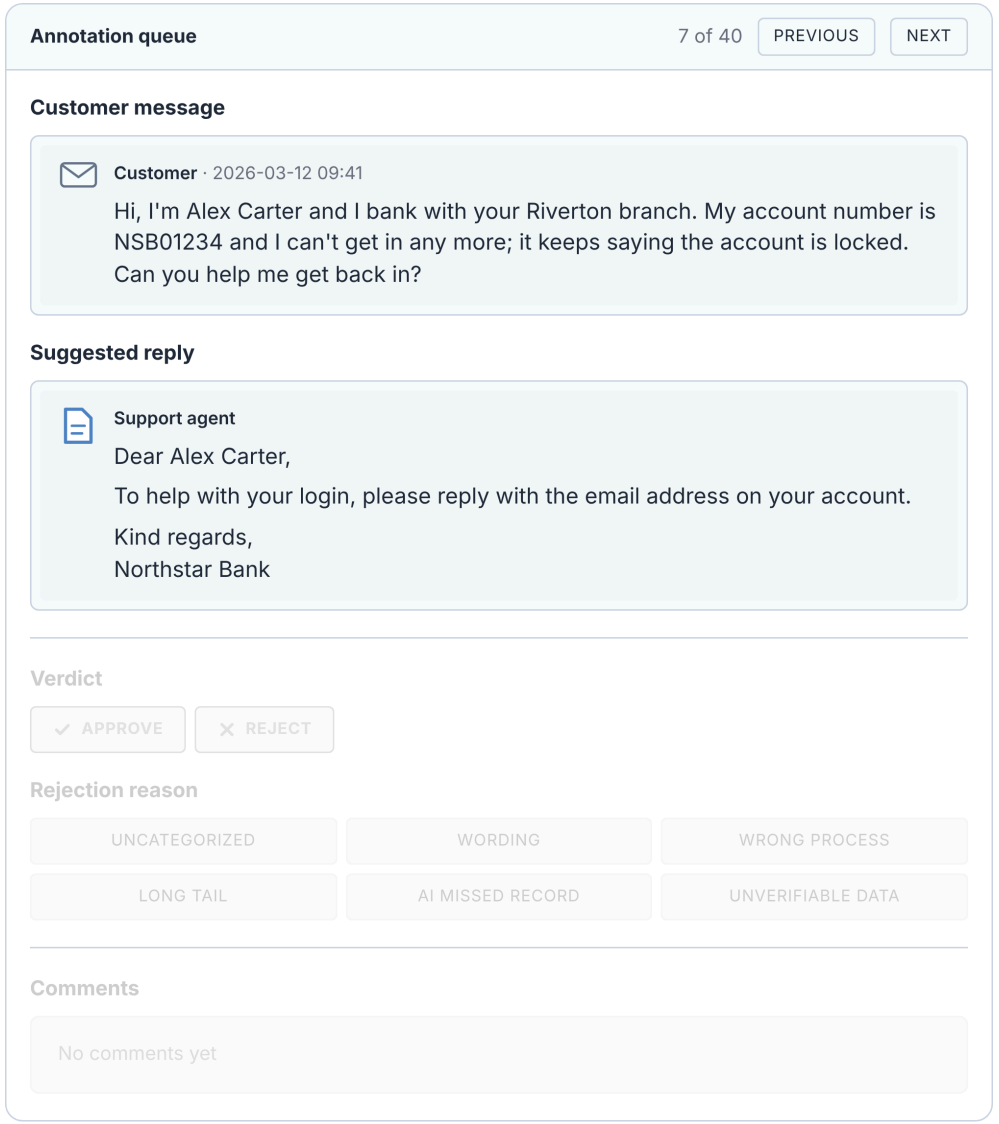


Shadow deployment: a draft for every real ticket, none sent



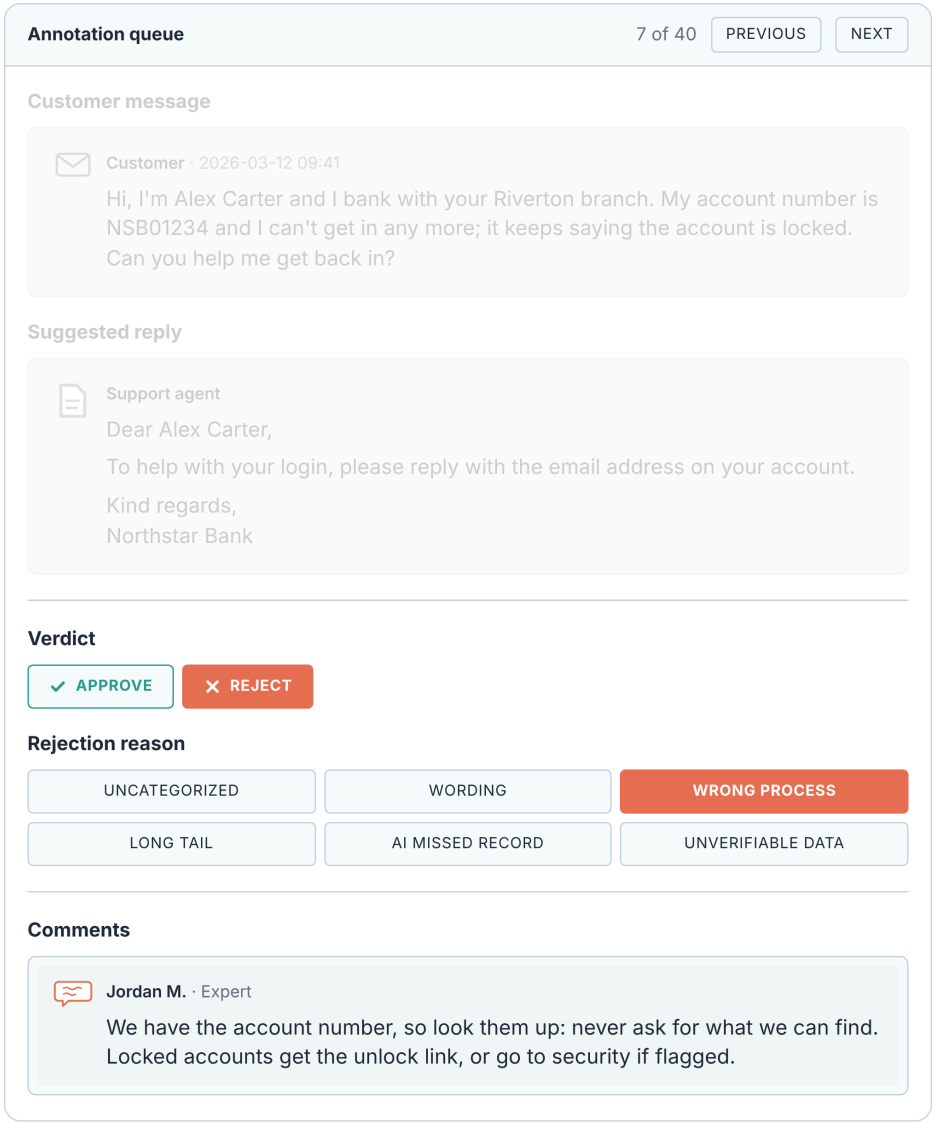
Step 1: experts grade

- The customer’s message and the proposed reply, nothing else.
- Graded when the expert picks up the ticket anyway.
- An approved draft is a reply they don’t have to write.



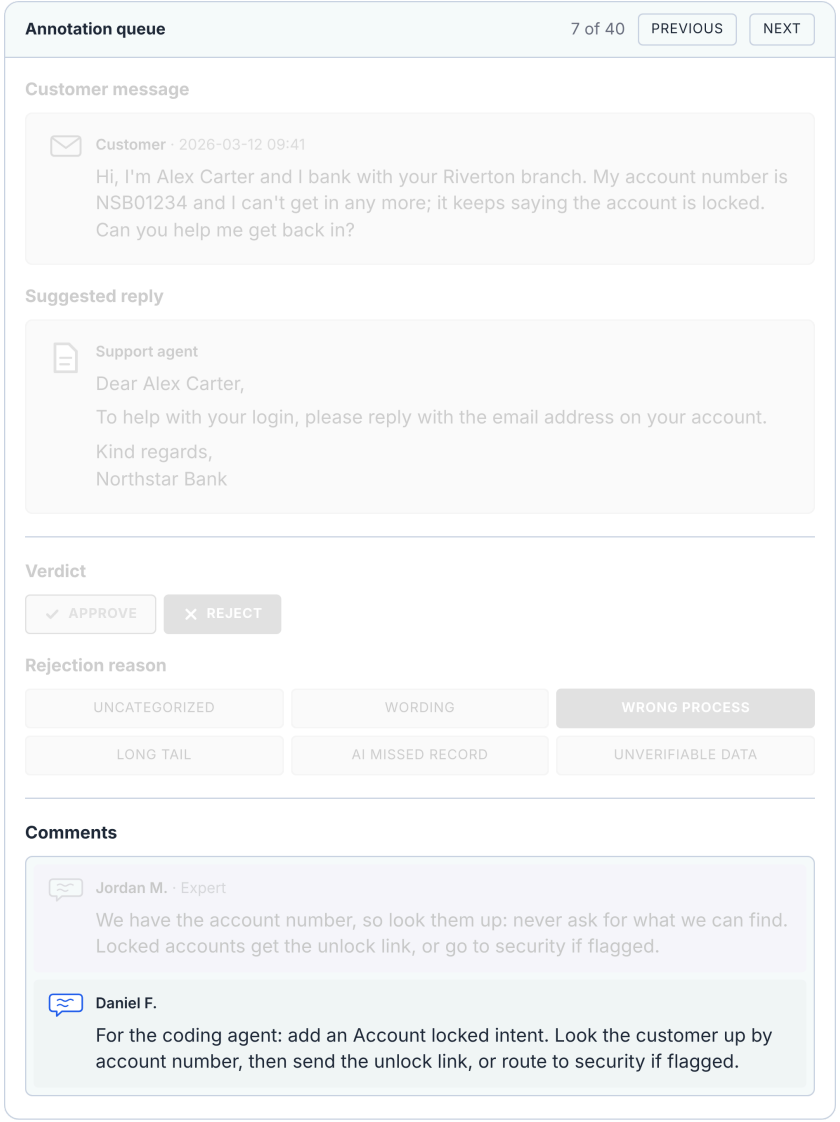
One rejection, two rules nobody had written down

- **Never ask for what we can find.**
- **Locked accounts get the unlock link, or go to security if flagged.**
- Grading is the check a coding agent gets from its tests, and how the rules get written down.



Step 2: I specify the fix

- In one sitting, for a batch of feedback.
- A plan for the coding agent next to the expert's comment: where the fix goes, what it does.
- Follow-up questions go in the tool, next to the ticket.



Designed so a fix fits in two sentences

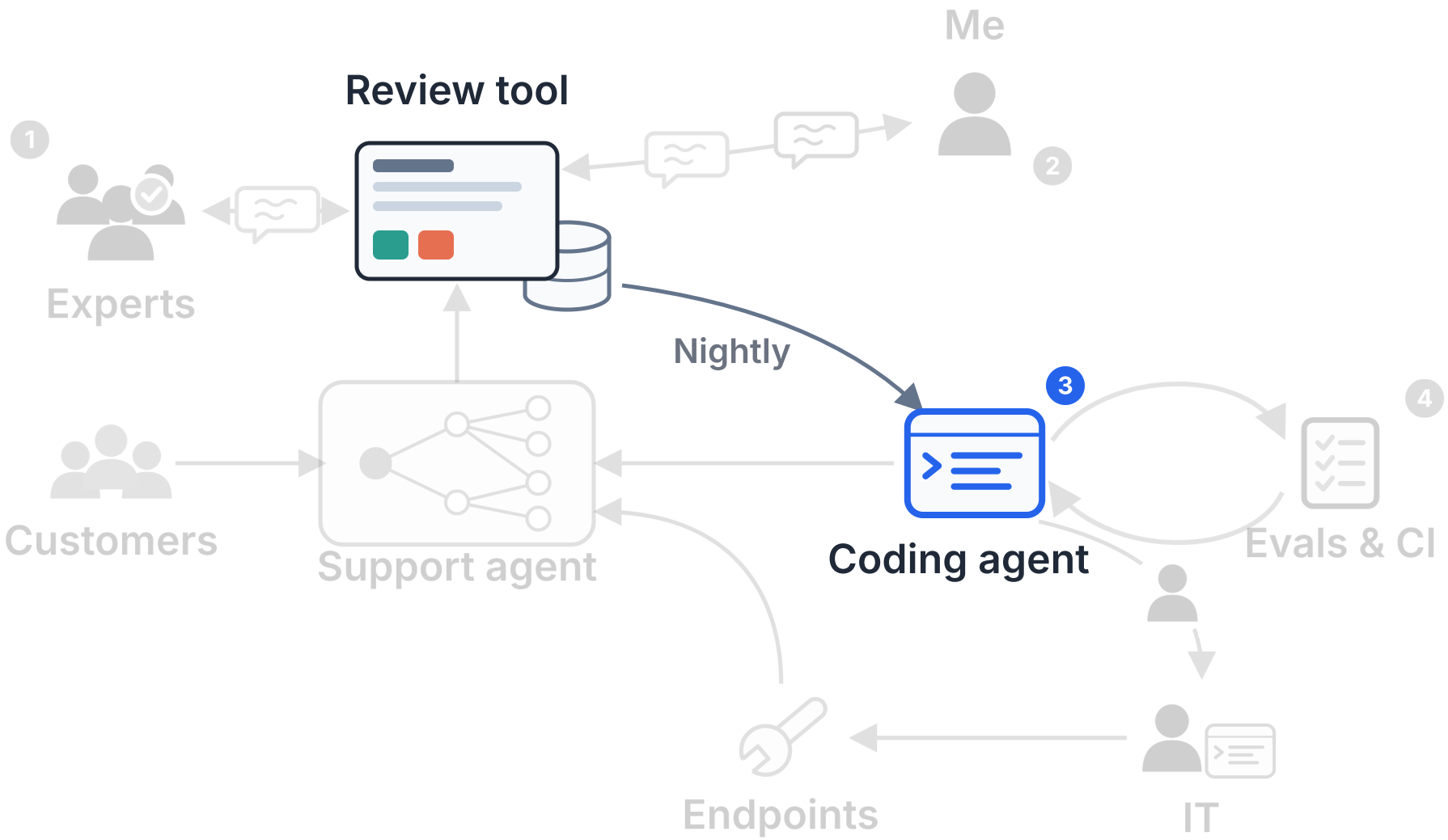
Most feedback becomes a change **in one place**:

- a new intent, a kind of request to recognise,
- a new branch under an intent it already knows,
- a second parent for an existing intent.

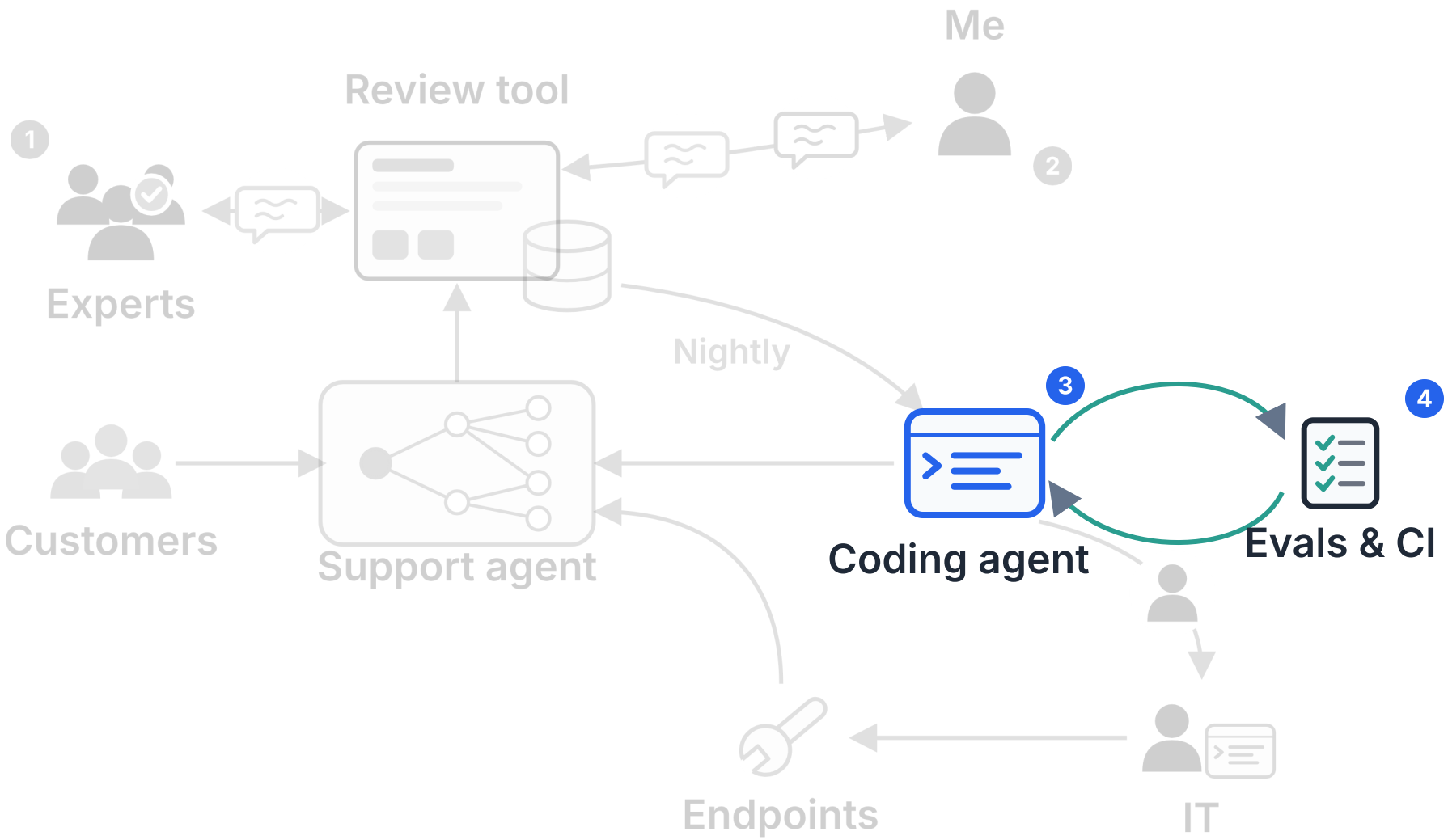
A change like that I can describe in two sentences and trust a coding agent to build.

The design work came first: which decisions code makes and which the model makes.

Step 3: the coding agent implements it overnight



Step 4: evals and CI check it



Evals: one trace per case, tied to a commit

- Eval cases live in git as JSONL, synced to Langfuse as datasets.
- Each run: one trace per case, with input, output and scores.
- Every trace carries its commit, branch and environment.

Separate from unit tests

Unit tests say whether the code works. Evals say how well the system answers, over time, run against run.

When an eval fails, agents triage it

1. **Discovery** lists the failing traces.
2. **Analyzer**, one trace at a time: finds the root cause, fixes it or leaves a note on the trace.
3. **Rerunner** re-runs the failed cases.

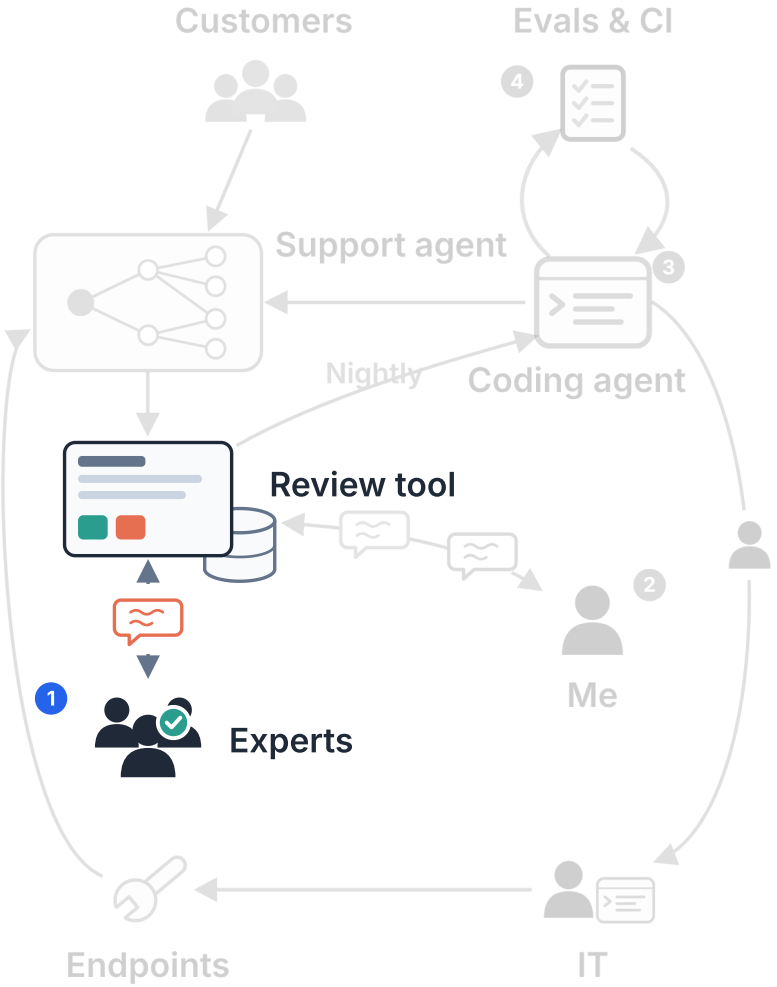
Root causes, each with its own fix

- Eval data wrong
- Mock doesn’t match the API
- Prompt wording
- Code bug
- Flaky: re-run, note it

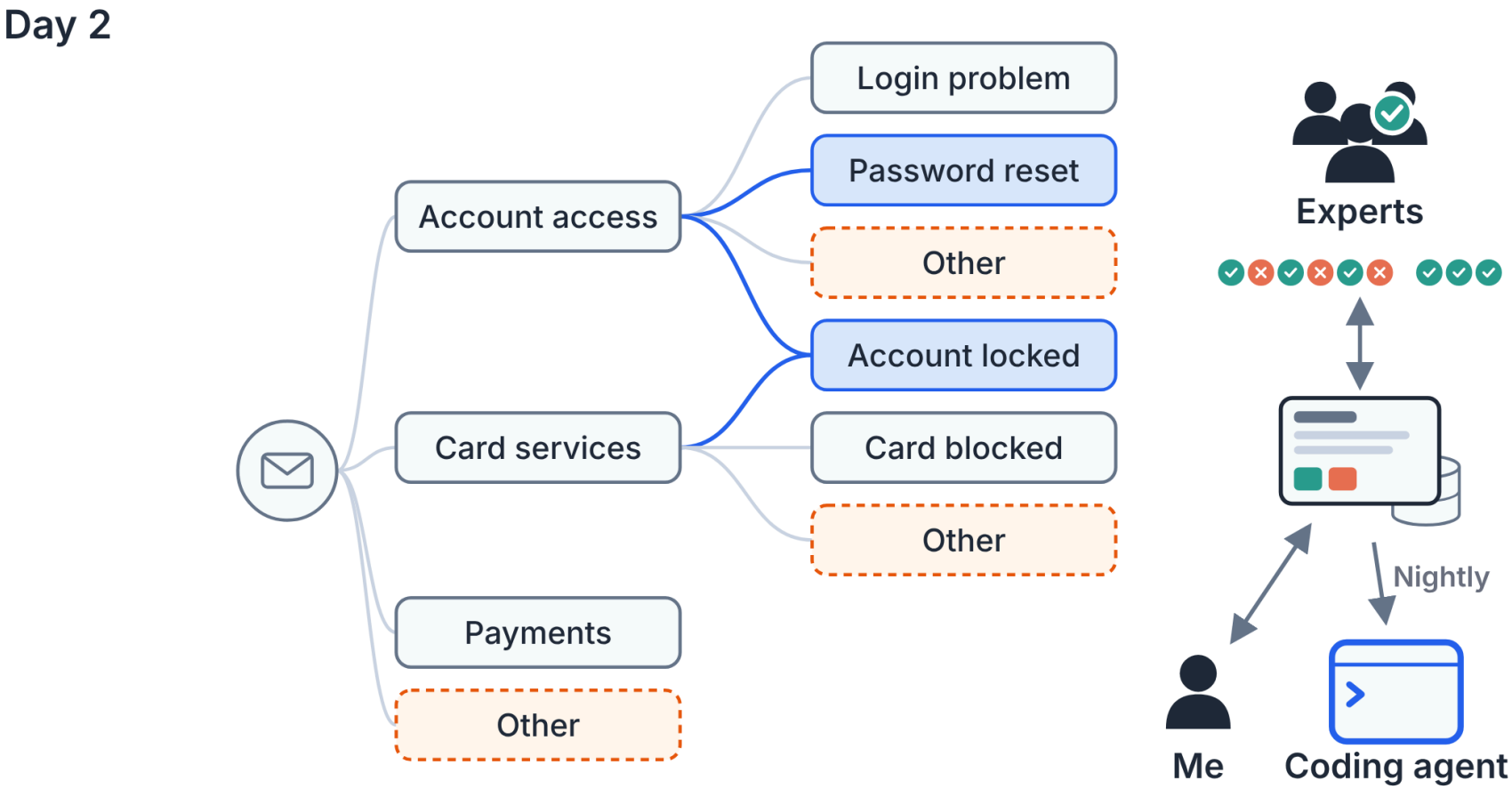
Never: editing a real customer message to make its test pass.

The factory checks against my plan; only an expert checks against the business

- The eval case is written from **my** plan.
- A green build says the code does what I asked, not that I asked for the right thing.
- So I resubmit the rejected ticket, and an expert checks the new draft.



After one round: the intent DAG grew overnight



One developer, ten experts, no direct messages

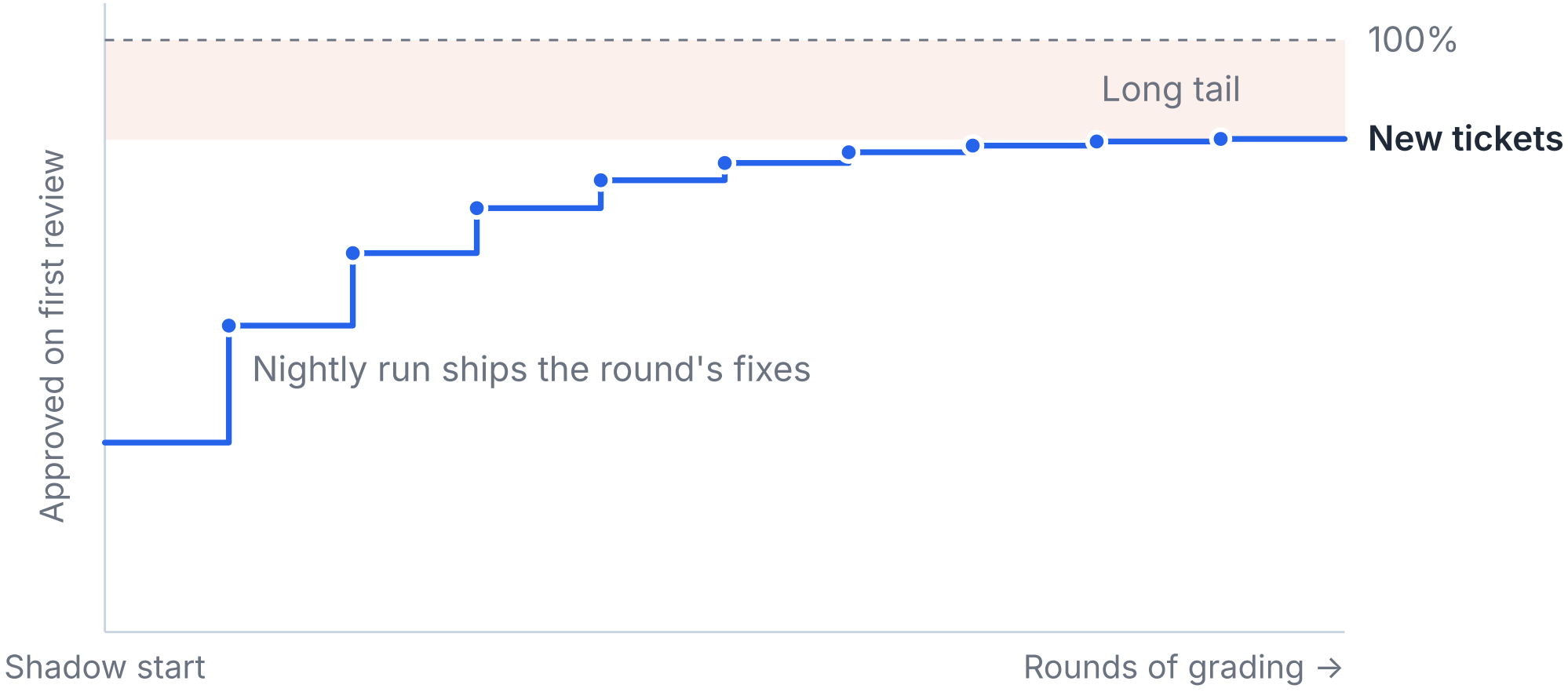
A direct message

- is about one ticket,
- gets lost a week later,
- needs an answer now.

A comment in the review tool

- stays with its ticket,
- is there when I batch the next fixes,
- waits until someone is ready.

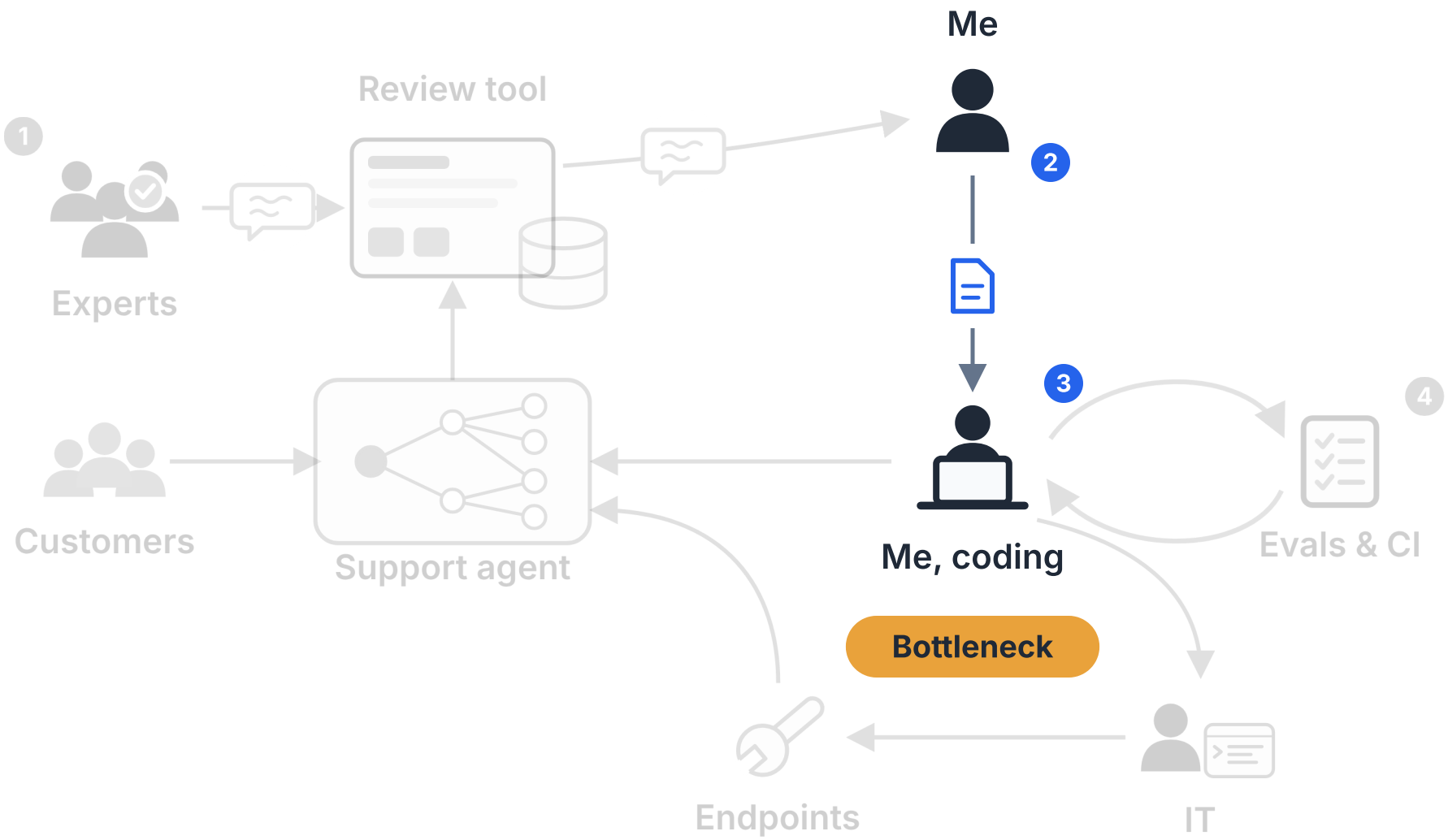
What the loop is built to raise



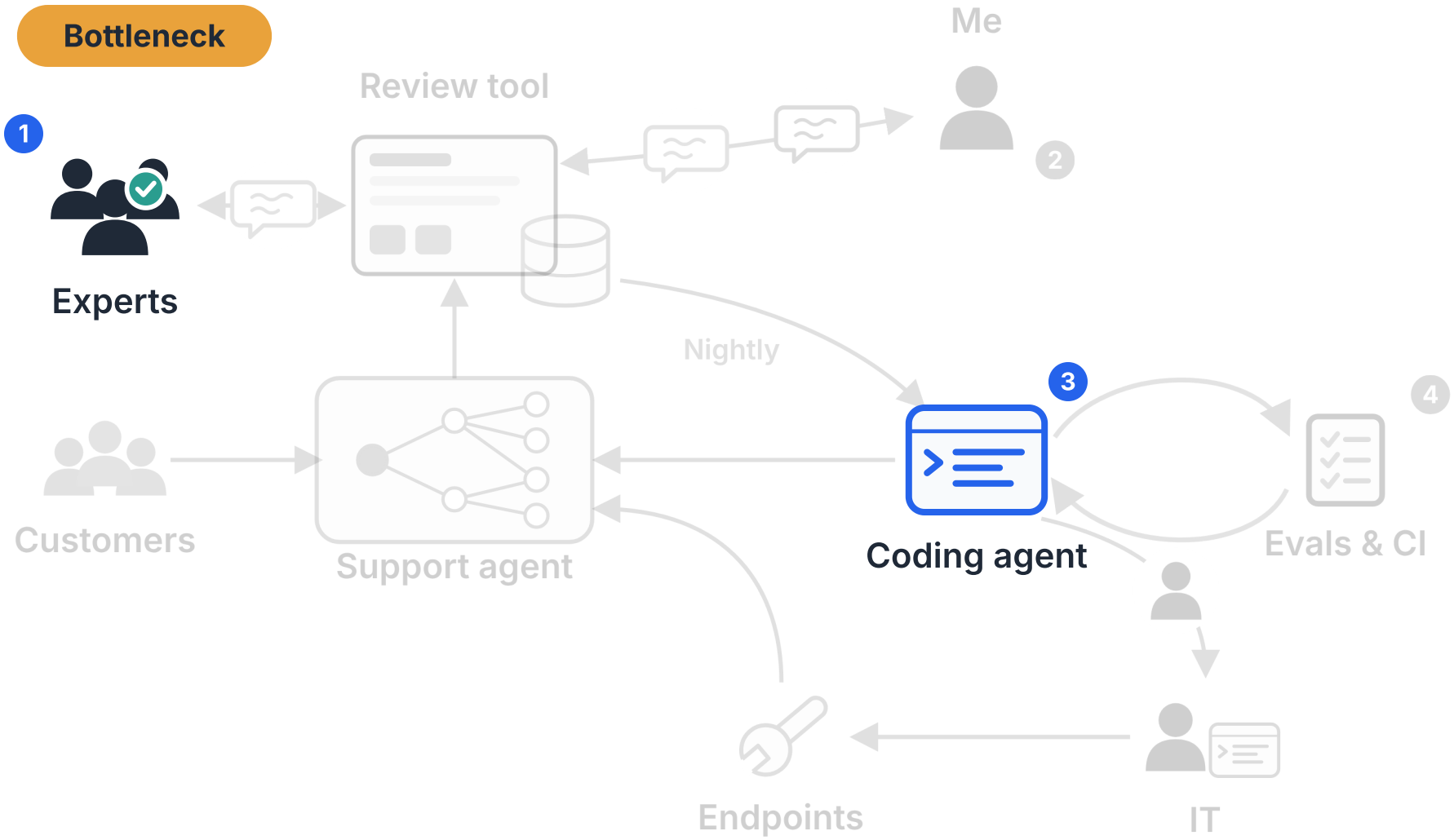
Every system has a bottleneck.
Fixing it doesn’t remove it; it moves it.

Eliyahu Goldratt, theory of constraints

Without coding agents: every fix goes through me



With coding agents: the experts' time sets the pace



Reviewers' time is a hard input

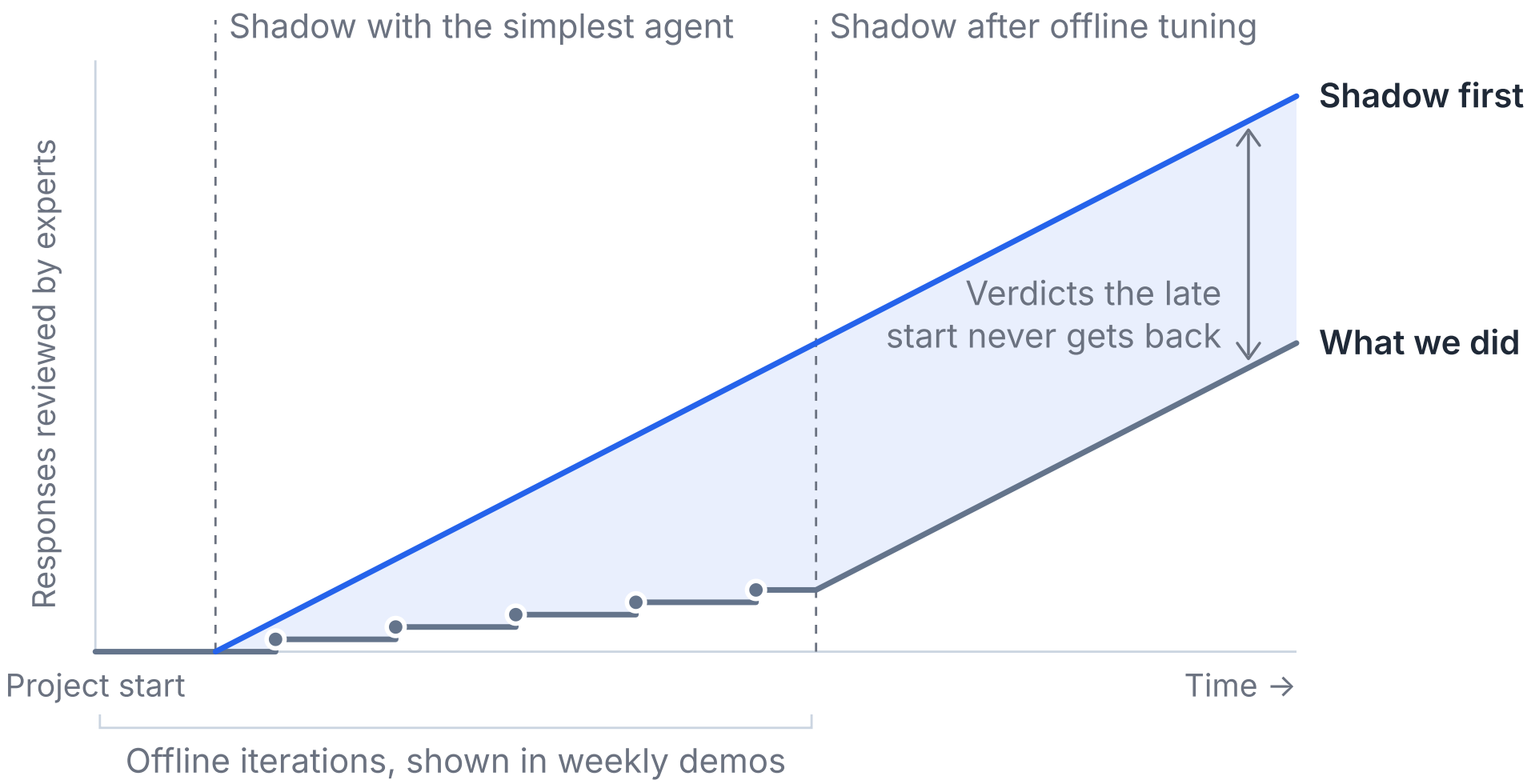
It cost them less than it might have

- They answered each ticket anyway.
- An approved draft was a reply they didn’t write.

It still cost them

- One more item in a full day.
- They may suspect they are training their own replacement.
- Engagement and sponsorship can’t be counted on.

Get into shadow deployment sooner



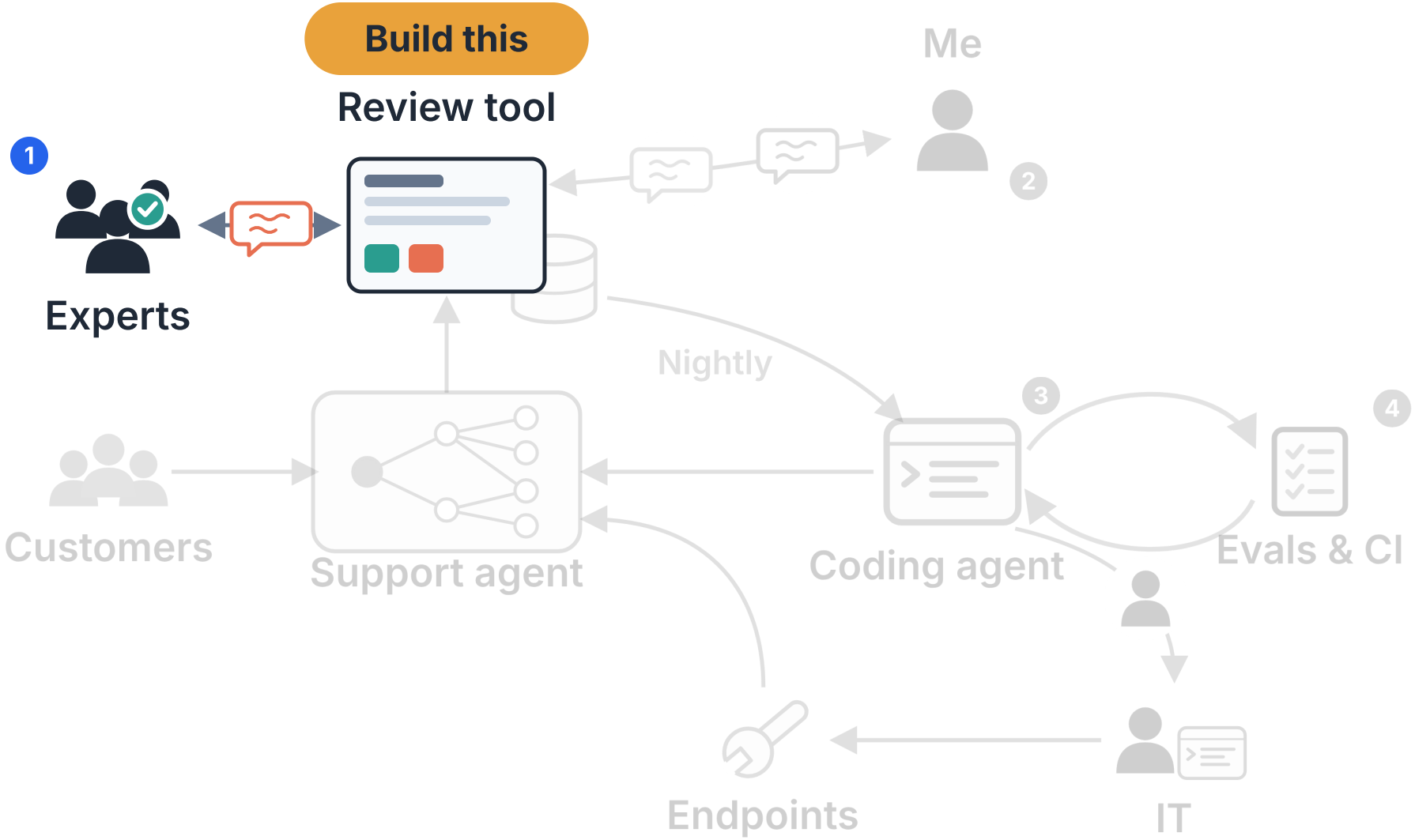
Replaying old tickets is the weaker signal

- A replayed ticket runs against **today's** records, not the customer's.
- A month later, Alex's account may already be unlocked.
- Only shadow deployment runs against live records.

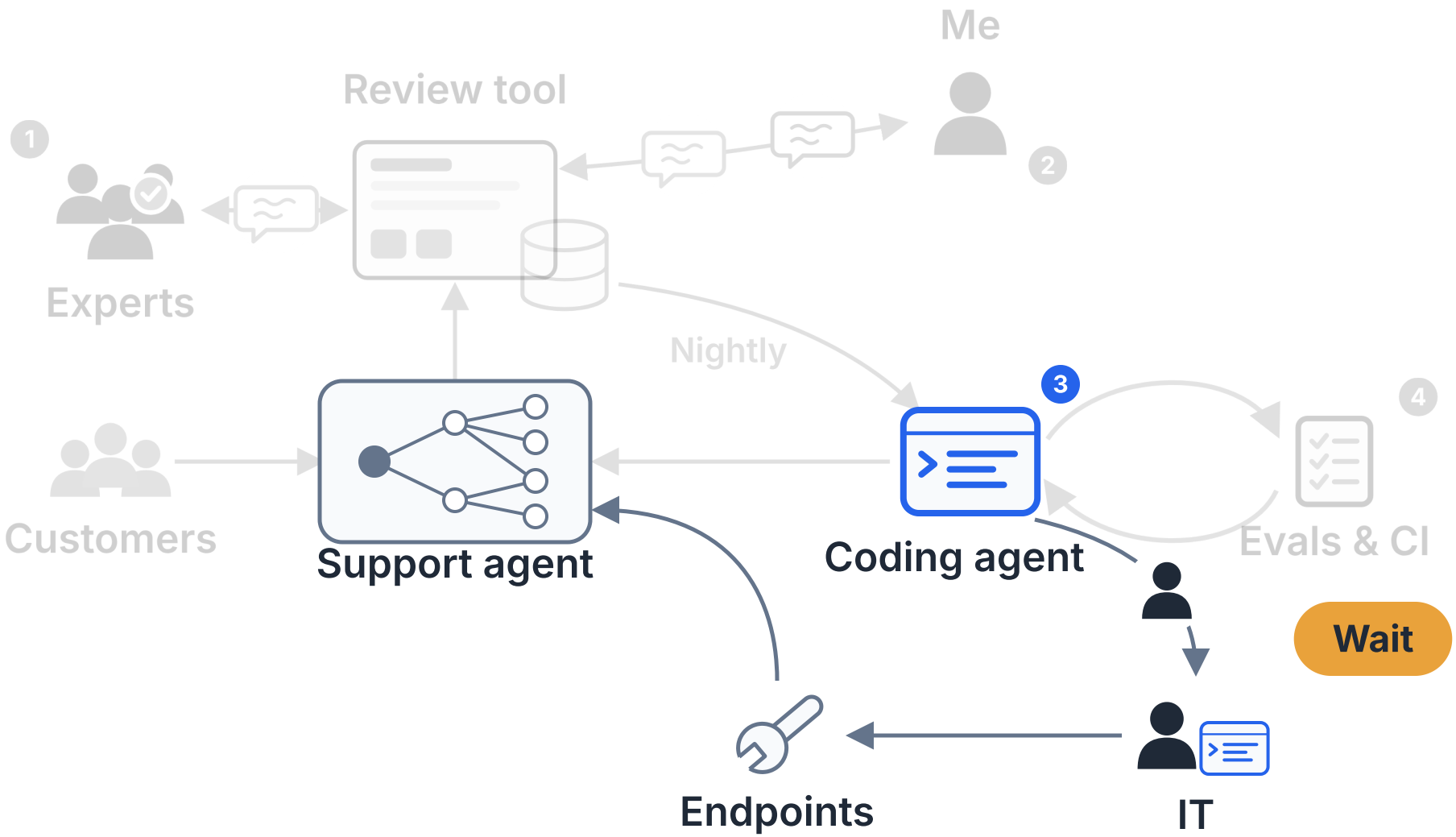
Replay is good for: checking that fixes stay fixed.

Replay is poor for: finding out what a good reply looks like.

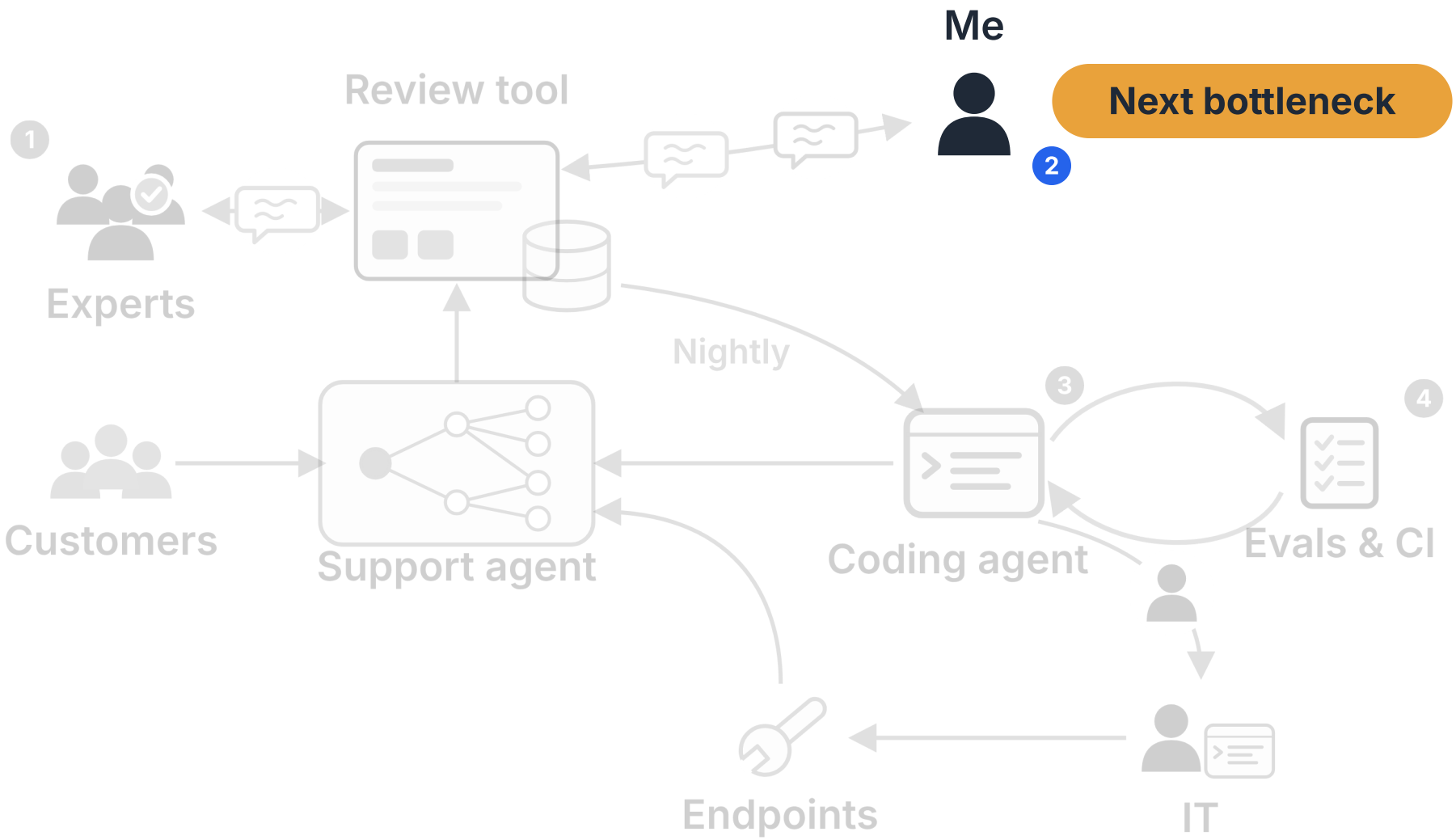
Build the review screen yourself, not the storage behind it



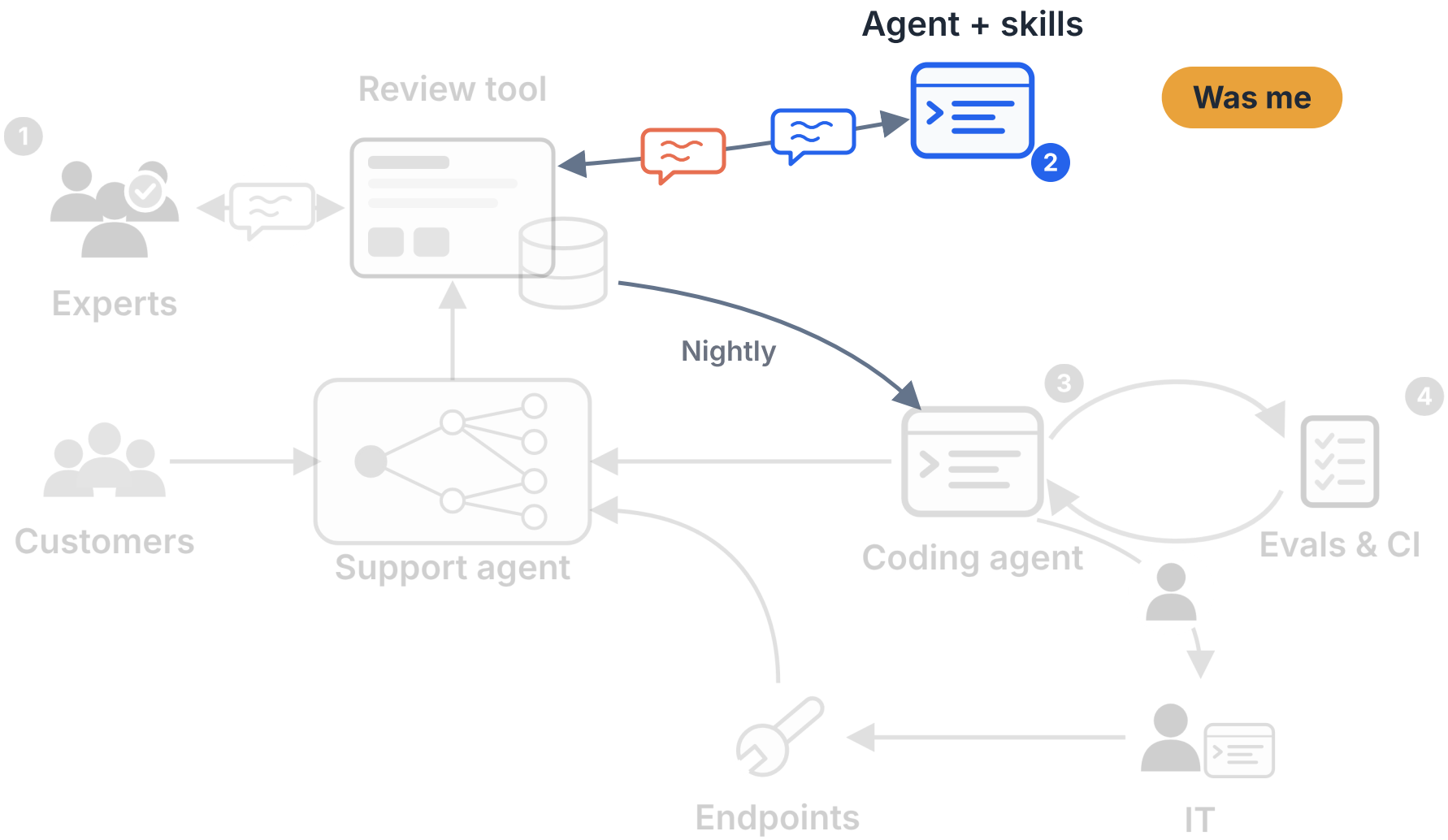
The other wait: endpoints



The next bottleneck: who translates the feedback



Could the agent write its own skills?



Start at the bottleneck

1. **Settle what the loop should improve**, and what that would save.
2. **Build the review screen and shadow deployment first**, with the simplest system that can draft a reply behind it.
3. **Then grow the rules from the grades**, not from a whiteboard.

Until someone who knows the business reads the output every day, every design decision is a guess.

danielfridljand.de/post/the-missing-test-suite